Bus Reservation System (Using Core Java & DSA Concepts)

RU-200-01-00029
Data structures and Algorithms

SESSION: 2026-27

Submitted By

Student name

Kirtan Yadav

Reg-No.-Ru-25-10686

Bachelor of Technology in Computer Science & Engineering with AI and ML in association with Google

Under the Guidance of

Mrs. Naina Singh

IBM SME and EXPERT

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

RUNGTA INTERNATIONAL SKILLS UNIVERSITY

BHILAI , CG

(October-2026)

◆ Abstract

The Bus Reservation System is a Java-based console application designed to manage bus ticket booking, cancellation and passenger records. The system allows users to view available buses, search a bus by its ID, sort buses by fare, book tickets, view bookings and cancel tickets. When a bus is full, passengers are automatically placed on a waiting list, and the first waiting passenger is given the seat as soon as a ticket is cancelled.

This project is needed to demonstrate how real-world reservation systems work in a simplified manner using programming concepts. Manual ticket booking is slow, error-prone and can lead to double booking of seats, so this system provides an automated way to allocate seats, generate PNR numbers and manage passenger data.

The application uses object-oriented programming concepts such as classes, objects, constructors, nested classes and methods. It also uses data structures from the Java Collections Framework: ArrayList to store buses and bookings, a boolean array to track seat availability, and a Queue (LinkedList) to manage the waiting list on a first-come, first-served basis. Searching (linear search) and sorting (Comparator-based sort by fare) are also implemented. Overall, the system simulates a basic bus reservation application and enhances understanding of Java programming concepts.

◆ Introduction

The Bus Reservation System is a console-based application developed using Java and object-oriented programming principles. It is designed to manage the reservation of seats on different bus routes and to maintain a record of every passenger booking.

In real-world scenarios, online and offline reservation systems play an important role in public transport, travel agencies and tourism. This project simulates such a system in a simplified way, helping users understand how seats are allocated, how tickets are identified using a PNR (Passenger Name Record) number, and how cancellations and waiting lists are handled.

The application provides features such as viewing all buses, searching a bus, sorting buses by fare, booking a ticket, viewing active bookings and cancelling a ticket. It also includes a menu-driven interface, making it user-friendly.

This project mainly focuses on applying core Java concepts like classes, objects, arrays, collections (ArrayList and Queue), searching, sorting, loops and conditional statements to solve a real-life problem efficiently.

◆ Objectives

- To display and manage the details of multiple buses (route, fare and seats)
- To book tickets with automatic seat allocation and unique PNR generation
- To avoid double booking by tracking seat availability
- To maintain a waiting list when a bus is full and serve it automatically on cancellation
- To allow searching of a bus by ID and sorting of buses by fare
- To cancel tickets and release seats for other passengers
- To reduce manual work and errors in ticket booking
- To apply object-oriented programming concepts and the Java Collections Framework

◆ Scope

This project is suitable for small-scale applications such as a basic ticket booking system for a local bus operator, and for educational purposes. It can be used in colleges or small organizations to understand how reservation systems work.

In the future, this system can be enhanced by integrating a database for permanent storage of buses and bookings, adding file handling to save and load data, adding input validation with exception handling, building a graphical user interface (GUI), adding user login and payment modules, and connecting with real-time online bus APIs.

◆ System Requirements

Hardware Requirements:

- Computer/Laptop with minimum 4GB RAM
- Processor: Intel i3 or above

Software Requirements:

- Java JDK (Java Development Kit)
- IDE: VS Code / Eclipse / IntelliJ IDEA
- Operating System: Windows/Linux/Mac

◆ Methodology

The working of the system follows a step-by-step approach:

1. The program starts, loads the sample buses and displays a menu to the user.
2. The user selects an option such as viewing buses, searching a bus, sorting by fare, booking, viewing bookings or cancelling a ticket.
3. When booking a ticket, the user enters the Bus ID and the passenger name.
4. The system searches the bus using linear search. If the bus does not exist, an error message is shown.
5. If a seat is free, the first available seat is allocated and a Booking object with a unique PNR is created and stored in an ArrayList.
6. If the bus is full, the passenger name is added to that bus's waiting list (Queue).
7. When a ticket is cancelled using its PNR, the booking is marked inactive and the seat is released.
8. If the waiting list is not empty, the first passenger in the queue is automatically given the released seat and a new PNR is generated.
9. The program continues until the user chooses to exit.

◆ Flowchart

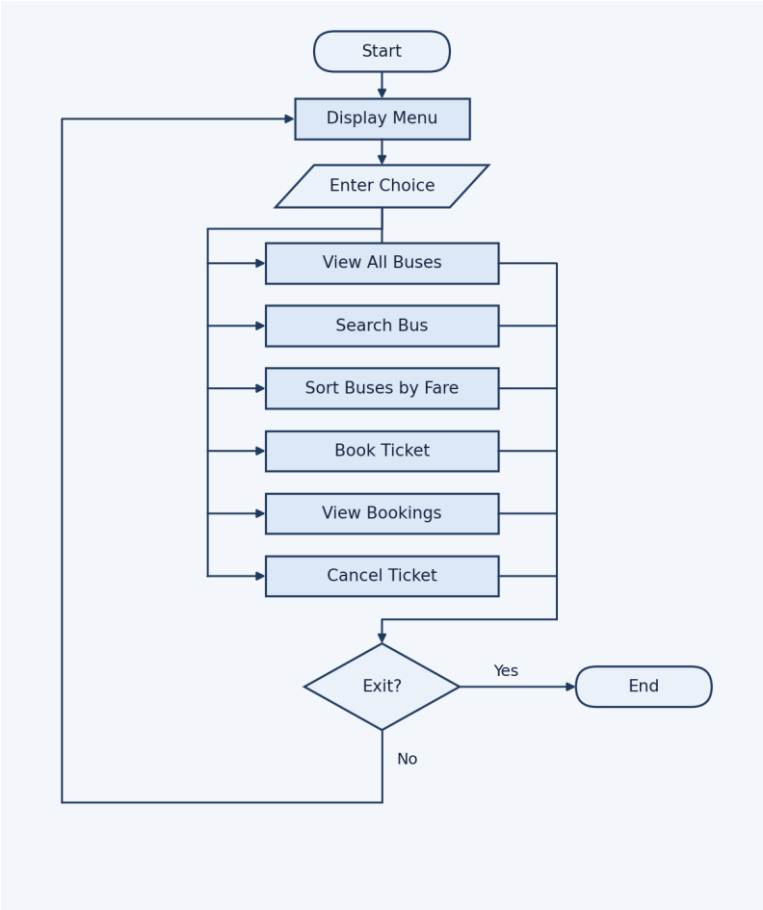


Figure 1: Flowchart of the Bus Reservation System

◆ Classes-Methods/Variables Used

Class: Bus (nested static class)

Variables:

- id (int)
- source (String)
- destination (String)
- fare (double)
- seats (boolean[]) – true means booked, false means free
- waitingList (Queue<String>)

Methods:

- Constructor Bus()
- availableSeats()

Class: Booking (nested static class)

Variables:

- pnr (int)
- busId (int)
- seatNo (int)
- passenger (String)

- active (boolean)

Methods:

- Constructor Booking()

Class: BusReservationSystem (main class)

Variables:

- ArrayList<Bus> buses
- ArrayList<Booking> bookings
- int nextPNR (starts from 1001)
- Scanner sc

Methods:

- addSampleBuses()
- displayBuses()
- searchBus()
- sortBusesByFare()
- bookTicket()
- displayBookings()
- cancelTicket()
- main()

◆ Implementation

The project consists of three classes: Bus, Booking and BusReservationSystem. Bus and Booking are nested static classes inside the main class BusReservationSystem.

The Bus class stores the bus ID, route (source and destination), fare and seat status. A boolean array represents the seats (false = free, true = booked), and a Queue implemented with LinkedList stores the names of passengers on the waiting list. The method availableSeats() counts the free seats.

The Booking class stores the details of one ticket: PNR, bus ID, seat number, passenger name and an active flag. The flag allows a cancelled ticket to stay in the record without being shown as an active booking.

The BusReservationSystem class is the main class that controls the program flow. It uses two ArrayLists to store all buses and all bookings. A menu-driven approach (do-while loop with a switch statement) is used to allow users to perform operations such as viewing, searching, sorting, booking and cancelling tickets.

Key logic used in the project:

- **Linear search:** searchBus() checks each bus one by one and returns the bus whose ID matches, or null if it is not found.
- **Sorting:** sortBusesByFare() uses ArrayList.sort() with Comparator.comparingDouble() to arrange buses in ascending order of fare.
- **Seat allocation:** bookTicket() loops over the seats array and allocates the first free seat.
- **Waiting list (Queue / FIFO):** offer() adds a passenger when the bus is full and poll() removes the first passenger when a seat becomes free.
- **PNR generation:** a static counter nextPNR is incremented for every new booking, so every PNR is unique.

◆ Source-code-Bus Reservation System

```
import java.util.*;

public class BusReservationSystem {

    // ArrayList stores all buses and bookings
    static ArrayList<Bus> buses = new ArrayList<>();
    static ArrayList<Booking> bookings = new ArrayList<>();

    static int nextPNR = 1001;
    static Scanner sc = new Scanner(System.in);

    // Bus class
    static class Bus {
        int id;
        String source, destination;
        double fare;
        boolean[] seats;
        Queue<String> waitingList = new LinkedList<>();

        Bus(int id, String source, String destination,
            double fare, int totalSeats) {
            this.id = id;
            this.source = source;
            this.destination = destination;
            this.fare = fare;
            seats = new boolean[totalSeats];
        }

        int availableSeats() {
            int count = 0;
            for (boolean seat : seats) {
                if (!seat) count++;
            }
            return count;
        }
    }

    // Booking class
    static class Booking {
        int pnr, busId, seatNo;
        String passenger;
        boolean active = true;

        Booking(int pnr, int busId, int seatNo, String passenger) {
            this.pnr = pnr;
            this.busId = busId;
            this.seatNo = seatNo;
            this.passenger = passenger;
        }
    }

    // Add sample buses
    static void addSampleBuses() {
        buses.add(new Bus(101, "Shahdol", "Bhopal", 650, 3));
        buses.add(new Bus(102, "Shahdol", "Jabalpur", 350, 3));
        buses.add(new Bus(103, "Bhopal", "Indore", 400, 3));
    }

    // Display buses
    static void displayBuses() {
        System.out.println("\n--- AVAILABLE BUSES ---");

        for (Bus b : buses) {
            System.out.println(
                "Bus ID: " + b.id +
                " | " + b.source + " -> " + b.destination +
```

```
                " | Fare: Rs." + b.fare +
                " | Seats: " + b.availableSeats()
            );
        }
    }

    // Linear search
    static Bus searchBus(int id) {
        for (Bus b : buses) {
            if (b.id == id) {
                return b;
            }
        }
        return null;
    }

    // Sort buses by fare
    static void sortBusesByFare() {
        buses.sort(Comparator.comparingDouble(b -> b.fare));
        System.out.println("\nBuses sorted by fare:");
        displayBuses();
    }

    // Book a ticket
    static void bookTicket() {
        System.out.print("Enter Bus ID: ");
        int id = sc.nextInt();
        sc.nextLine();

        Bus bus = searchBus(id);

        if (bus == null) {
            System.out.println("Bus not found!");
            return;
        }

        System.out.print("Enter passenger name: ");
        String name = sc.nextLine();

        if (bus.availableSeats() == 0) {
            bus.waitingList.offer(name);
            System.out.println("Bus is full. Added to waiting list.");
            return;
        }

        int seatNo = -1;

        for (int i = 0; i < bus.seats.length; i++) {
            if (!bus.seats[i]) {
                bus.seats[i] = true;
                seatNo = i + 1;
                break;
            }
        }

        Booking booking =
            new Booking(nextPNR++, bus.id, seatNo, name);

        bookings.add(booking);

        System.out.println("\nBooking successful!");
        System.out.println("Passenger: " + name);
        System.out.println("Seat Number: " + seatNo);
        System.out.println("PNR: " + booking.pnr);
        System.out.println("Fare: Rs." + bus.fare);
    }

    // Display bookings
    static void displayBookings() {
```

```
        System.out.println("\n--- BOOKING DETAILS ---");
        boolean found = false;

        for (Booking b : bookings) {
            if (b.active) {
                System.out.println(
                    "PNR: " + b.pnr +
                    " | Passenger: " + b.passenger +
                    " | Bus ID: " + b.busId +
                    " | Seat: " + b.seatNo
                );
                found = true;
            }
        }

        if (!found) {
            System.out.println("No active bookings found.");
        }
    }

    // Cancel booking and serve waiting list
    static void cancelTicket() {
        System.out.print("Enter PNR: ");
        int pnr = sc.nextInt();

        for (Booking b : bookings) {
            if (b.pnr == pnr && b.active) {
                b.active = false;

                Bus bus = searchBus(b.busId);
                bus.seats[b.seatNo - 1] = false;

                System.out.println("Ticket cancelled successfully.");

                // Queue: first passenger gets the available seat
                if (!bus.waitingList.isEmpty()) {
                    String name = bus.waitingList.poll();

                    bus.seats[b.seatNo - 1] = true;

                    Booking newBooking = new Booking(
                        nextPNR++, bus.id, b.seatNo, name
                    );
                    bookings.add(newBooking);

                    System.out.println(
                        name + " moved from waiting list to seat " +
                        b.seatNo + ". New PNR: " + newBooking.pnr
                    );
                }
                return;
            }
        }

        System.out.println("Active booking not found!");
    }

    // Main method
    public static void main(String[] args) {
        addSampleBuses();

        int choice;

        do {
            System.out.println("\n================================");
            System.out.println("     BUS RESERVATION SYSTEM");
            System.out.println("================================");
            System.out.println("1. View All Buses");
            System.out.println("2. Search Bus");
```

```
            System.out.println("3. Sort Buses by Fare");
            System.out.println("4. Book Ticket");
            System.out.println("5. View Bookings");
            System.out.println("6. Cancel Ticket");
            System.out.println("0. Exit");
            System.out.print("Enter your choice: ");

            choice = sc.nextInt();

            switch (choice) {
                case 1:
                    displayBuses();
                    break;

                case 2:
                    System.out.print("Enter Bus ID: ");
                    int id = sc.nextInt();
                    Bus bus = searchBus(id);

                    if (bus != null) {
                        System.out.println(
                            bus.id + ": " + bus.source + " -> " +
                            bus.destination + ", Fare: Rs." + bus.fare
                        );
                    } else {
                        System.out.println("Bus not found!");
                    }
                    break;

                case 3:
                    sortBusesByFare();
                    break;

                case 4:
                    bookTicket();
                    break;

                case 5:
                    displayBookings();
                    break;

                case 6:
                    cancelTicket();
                    break;

                case 0:
                    System.out.println("Thank you for using the system!");
                    break;

                default:
                    System.out.println("Invalid choice!");
            }

        } while (choice != 0);

        sc.close();
    }
}
```

◆ Sample Input / Output

1. Main menu and View All Buses

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\BTECH G1\JAVA-Project> javac BusReservationSystem.java ; java BusReservationSystem

================================
     BUS RESERVATION SYSTEM
================================
1. View All Buses
2. Search Bus
3. Sort Buses by Fare
4. Book Ticket
5. View Bookings
6. Cancel Ticket
0. Exit
Enter your choice: 1

--- AVAILABLE BUSES ---
Bus ID: 101 | Shahdol -> Bhopal | Fare: Rs.650.0 | Seats: 3
Bus ID: 102 | Shahdol -> Jabalpur | Fare: Rs.350.0 | Seats: 3
Bus ID: 103 | Bhopal -> Indore | Fare: Rs.400.0 | Seats: 3
```

Figure 2: Menu and list of available buses

2. Search Bus and Sort Buses by Fare

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

Enter your choice: 2
Enter Bus ID: 102
102: Shahdol -> Jabalpur, Fare: Rs.350.0

Enter your choice: 3

Buses sorted by fare:

--- AVAILABLE BUSES ---
Bus ID: 102 | Shahdol -> Jabalpur | Fare: Rs.350.0 | Seats: 3
Bus ID: 103 | Bhopal -> Indore | Fare: Rs.400.0 | Seats: 3
Bus ID: 101 | Shahdol -> Bhopal | Fare: Rs.650.0 | Seats: 3
```

Figure 3: Searching a bus and sorting buses by fare

3. Book Ticket and Waiting List

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

Enter your choice: 4
Enter Bus ID: 102
Enter passenger name: Kirtan

Booking successful!
Passenger: Kirtan
Seat Number: 1
PNR: 1001
Fare: Rs.350.0

Enter your choice: 4
Enter Bus ID: 102
Enter passenger name: Sneha
Bus is full. Added to waiting list.
```

Figure 4: Booking a ticket; when the bus is full the passenger joins the waiting list

4. Cancel Ticket and Exit



Figure 5: Cancelling PNR 1002 gives the seat to the first waiting passenger

◆ Gantt Chart

Task	Duration
Planning & Design	1 Day
Coding (Classes & Logic)	2 Days
Testing	1 Day
Searching, Sorting & Waiting List Integration	1 Day
Documentation	1 Day
Total Estimated Time	**6 Days**

◆ Conclusion

The Bus Reservation System project successfully demonstrates the use of Java and object-oriented programming concepts in solving a real-world problem. It provides a functional system to view, search, sort, book and cancel bus tickets efficiently, while automatically handling waiting-list passengers through a queue.

The project helped in understanding key concepts such as classes and objects, nested classes, arrays, the ArrayList and Queue collections, linear search, sorting with a Comparator and menu-driven programming. It also improved logical thinking and problem-solving skills. Overall, this system serves as a strong foundation for developing more advanced applications in the future.